

MASTER DOCUMENT: Cloud Storage Panel + R2 Migration System + Multi-Account Control Plane

Version: 1.0

Author: ChatGPT + Nadith

Status: Approved

Purpose: Fully define every detail of the project—backend, frontend, agents, APIs, features, flows, UX, reliability, accuracy, and documentation.

1. PROJECT OVERVIEW

The goal is to build a full cloud storage solution with:

- Hosted storage panel
- Multi-Cloudflare R2 account management
- Custom URL endpoints controlled by the panel
- User management, quotas, permissions
- File browser (Google Drive–style)
- Multi-bucket operations
- Full migration engine (20+ TB)
- Migration agents across multiple VPS nodes
- Auto-migration every 20 days
- Strong data integrity guarantees (practically 100%)
- Full documentation system embedded in the website

The system behaves as a “cloud storage provider” on top of Cloudflare R2, abstracting raw R2 complexity behind a modern, user-friendly panel and API.

2. CORE COMPONENTS

1) Web UI Panel

- Built with React / Next.js / Tailwind / shadcn UI.
- Provides full admin and user experience: buckets, files, links, migrations, accounts.

2) Backend API

- Node.js backend (NestJS or similar).
- Provides REST/JSON API for the UI and agents.
- Handles authentication, authorization, validation, scheduling, migrations, and verification.

3) Database (PostgreSQL)

- Stores user accounts, Cloudflare accounts, buckets, migration jobs, permissions, and audit logs.

4) Job Queue (Redis + BullMQ or equivalent)

- Manages long-running jobs (migrations, verifications, indexing, scheduled tasks).

5) Migration Agents

- Lightweight clients installed on VPS servers.
- Poll jobs from backend, perform high-throughput R2-to-R2 migrations using S3-compatible APIs.

6) Cloudflare R2 Integration

- Uses S3-compatible endpoints for file operations (list, upload, copy, delete).
- Uses Cloudflare REST APIs for R2 management (buckets, custom domains, public URLs).

7) File Browser & Endpoint Layer

- Implements a folder-like view on top of R2 object keys (prefix-based).
- Exposes panel-owned URLs for HTML previews and raw file access.

8) Search & Indexing

- Indexes objects metadata (name, size, type, path, owner) to support powerful search and filters.

9) Documentation System

- Built-in documentation pages explaining how to use the panel, endpoints, and features.

3. FEATURE LIST (HIGH-LEVEL)

A. Cloudflare Account Management

- Add multiple Cloudflare accounts.
- Store encrypted R2 access keys and API tokens.
- Mark accounts as Active or Disabled.
- Configure automatic migration from one account to another.
- Auto-migration interval (default 20 days).
- Manual status toggle and migration trigger.

B. Bucket Management

- List buckets per Cloudflare account.
- Create new buckets.
- Delete buckets with safety checks.
- Configure public dev URLs and custom domains.
- Show approximate size and object counts.
- Attach buckets to user-facing “spaces.”

C. File Browser

- Tree and breadcrumb navigation (folders simulated via prefixes).
- List/grid views.
- Drag & drop uploads with progress.
- Bulk operations: move, copy, delete, rename.
- Preview for images, video, audio, and text where possible.
- Trash for soft-delete.
- Versioning support via naming or R2 versions (if enabling).

D. File Sharing & Links

- Panel-owned URLs for HTML file views.
- Raw download/stream URLs.
- Time-limited signed URLs for private files.
- Public share links with optional password and expiry.

E. User Management & Permissions

- Add/edit/remove users.

- Roles: Admin, Manager, User, Viewer.
- Storage quota per user (e.g. 500 GB, 2 TB).
- Bucket-level access rules (all buckets vs specific buckets, read/write/admin).

F. Migration Engine

- Direct mode using custom agents on VPS.
- Cloudflare mode using R2 migration tools where applicable.
- Handles large datasets (>20 TB) and many objects.
- Supports parallelism, retries, and resumable state.
- Performs post-migration verification (size/ETag).
- Supports multi-agent parallel migrations.

G. Migration Agents (VPS Clients)

- Prebuilt binaries or Docker images.
- Installed via simple installer script.
- Register with panel using token.
- Poll for jobs, execute R2-to-R2 transfers, and report progress.
- Automatically reconnect and resume work.

H. Auto-Migration Every 20 Days

- Scheduler finds accounts due for migration.
- Creates jobs from source account to configured target account.
- Runs verification after migration.
- Marks old account disabled and new account active.
- Updates references in DB to use new account.

I. URL Endpoint System (Panel-controlled URLs)

- HTML preview: `/f/:accountSlug/:bucket/:path`
- Raw access: `/raw/:accountSlug/:bucket/:path`
- Public share: `/p/:token`
- Supports access control, logging, and optional redirect to R2 signed URLs.

J. Search & Filters

- Search files by name, prefix, path, bucket, account.
- Filter by type, size range, date range, owner, visibility.
- Results are paginated for large data sets.

K. Audit Logging & Activity Feed

- Tracks user activity: login, upload, delete, share, migration actions.
- Audit log per user, per space, and global logs for admins.

L. Accuracy & Verification

- Every migration job optionally runs a verification stage.
- Compares object counts, total bytes, and optionally ETags.
- Can run repair jobs for mismatches.
- Does not mark migration as final until verification passes.

4. SYSTEM ARCHITECTURE

Core flows:

1. UI -> Backend

- The frontend calls backend REST endpoints for all actions (login, list files, create migration, etc.).

2. Backend -> R2

- For file operations: uses S3-compatible R2 endpoints with per-account credentials.
- For bucket and configuration operations: uses Cloudflare REST APIs.

3. Backend -> Agents

- Agents communicate via backend endpoints (polling or lightweight WebSocket if desired).
- Backend assigns jobs, receives progress, and updates migration status.

4. Backend -> Database & Queue

- All state is persisted in PostgreSQL.
- Long jobs scheduled via a queue system (Redis + BullMQ).

5. Auto-Migration

- A scheduler process periodically checks for accounts where last migration is older than the configured interval (e.g. 20 days).
- Creates or queues appropriate migration jobs.

5. DATABASE SCHEMA (OVERVIEW)

Key tables (fields abbreviated):

Table: users

- id, email, password_hash, role, storage_limit_bytes, is_active, created_at

Table: cf_accounts

- id, name, account_id, status, auto_migrate_enabled, migrate_interval_days, migrate_to_account_id, last_migrated_at, r2_access_key_id_enc, r2_secret_key_enc, api_token_enc, created_by_user_id

Table: buckets

- id, cf_account_id, name, region, public_enabled, custom_domain, size_cache_bytes, object_count_cache, created_at

Table: bucket_permissions

- id, user_id, bucket_id, access_level ("read", "write", "admin")

Table: spaces

- id, name, owner_user_id, bucket_id, base_prefix, used_bytes

Table: objects (for indexing/search)

- id, cf_account_id, bucket_id, key, size_bytes, content_type, owner_user_id, space_id, is_public, created_at, updated_at

Table: migration_jobs

- id, mode ("direct", "cloudflare"), status, source_account_id, dest_account_id, source_bucket_id (optional), dest_bucket_id (optional), total_bytes, transferred_bytes, total_objects, transferred_objects, started_at, finished_at, error_summary

Table: migration_events

- id, migration_job_id, timestamp, level, message, details_json

Table: agents

- id, name, token_enc, status, region, capacity, last_heartbeat_at

Table: audit_logs

- id, user_id, action, target_type, target_id, timestamp, details_json

6. CLOUDFLARE ACCOUNT MANAGEMENT

Features:

- Add Cloudflare accounts by providing:
 - account_id
 - R2 access key & secret
 - Optional Cloudflare API token
- Accounts are persisted in cf_accounts with sensitive fields encrypted.
- Each account has:
 - status: "active" or "disabled"
 - auto_migrate_enabled: boolean
 - migrate_interval_days: integer (default 20)
 - migrate_to_account_id: account where data will be migrated to
 - last_migrated_at: datetime
 - UI option to manually mark as active/disabled
 - UI option to run migration now

Rules:

- Only "active" accounts are used for new writes / operations.
- After a full successful migration from account A to account B, account A is typically set to "disabled".
- Admins can override and re-enable accounts when needed.

7. BUCKET & FILE BROWSER

Bucket Management:

- UI page: "Accounts" -> select account -> see list of buckets.
- Buckets show:
 - Name
 - Region
 - Public or private indicators
 - Approximate size and object count
- Actions:
 - Create bucket
 - Delete bucket (with confirmation)
 - Enable/disable public dev URL or custom domain

File Browser:

- Folder view simulated with R2 object key prefixes.
- For prefix "a/b/c/", list objects with that prefix, with delimiter "/" to group "subfolders".
- Supports pagination on list operations.
- Shows icon, name, size, type, date, owner, and link options.

File Operations:

- Upload:
 - Client uploads via backend or directly to R2 with pre-signed URLs.
 - Large files use multipart uploads.
- Rename:
 - Implemented as copy to new key + delete old key.
- Move / Copy:
 - Same as rename but with different prefix or bucket.
- Delete:
 - Soft-delete: move object to a dedicated ".trash/" prefix
 - Permanent delete: remove from R2 and index.
- Preview:
 - Images: render in browser.
 - Video/audio: stream via /raw endpoint with appropriate headers.
 - Text: show in on-page viewer.

8. FILE LINKS & ENDPOINTS

The panel provides user-facing links based on its own domain.

Endpoint types:

1) HTML file view

- URL: /f/:accountSlug/:bucket/*objectPath
- Behavior:
 - Checks user or link token permissions.
 - Renders viewer page with metadata and preview.
 - Uses backend to fetch metadata and either proxy or redirect to R2 URL.

2) Raw file download/stream

- URL: /raw/:accountSlug/:bucket/*objectPath
- Behavior:
 - Authorization check.
 - Option 1: directly proxy data from R2 via backend stream.
 - Option 2: generate a short-lived signed R2 URL and redirect.

3) Public share link

- URL: /p/:shareToken
- Behavior:
 - shareToken maps to file + share config (optional password, expiry, download limits).
 - If token valid and active:
 - Either show HTML view or redirect to /raw URL according to share mode.

Access Modes:

- Private: only authenticated users with access can view.
- Public link: unguessable share link; good for public file sharing.
- Time-limited signed URLs: valid until expiration; used for private access without login.

9. USER MANAGEMENT, STORAGE LIMITS & PERMISSIONS

User Roles:

- Admin
 - Full access to all accounts, buckets, users, migrations, agents, settings.
- Manager
 - Can manage users within limits, see most buckets, start migrations, etc. (configurable).
- User
 - Restricted to their assigned buckets/spaces.
- Viewer
 - Read-only access per assigned buckets/spaces.

Storage Limits:

- Each user has storage_limit_bytes (nullable for unlimited).
- Each user owns one or more spaces (logical views into R2).
- Each upload increments the space used_bytes; delete decrements.
- System enforces quota before each upload.

Bucket Permissions:

- bucket_permissions assigned per user:
 - access_level values:
 - "read": list and read files
 - "write": upload, rename, delete own files
 - "admin": can manage bucket configuration (for that bucket)

Admins bypass bucket_permissions and see all accounts/buckets by default.

10. MIGRATION ENGINE (DIRECT + CLOUDFLARE)

Migration Modes:

Mode: "direct"

- Uses custom migration agents running on VPS servers.
- Agents pull jobs from backend and perform S3-based copy operations.
- Suitable for R2-to-R2 cross-account migrations and very large data sets.

Mode: "cloudflare"

- Uses Cloudflare migration tools (e.g., Super Slurper) when possible.
- Backend helps configure and track such migrations.
- After migration, backend verifies data by scanning destination via API.

Migration Job Lifecycle:

- Created with:
 - source_account_id, dest_account_id
 - optional bucket/prefix filters
 - mode ("direct" or "cloudflare")
 - options (concurrency, bandwidth limits, verification options)
- Status transitions:
 - pending -> running -> verifying -> completed or failed
- Verification phase:
 - Compares object lists between source and destination.
 - Checks sizes and optionally ETags/hash.
 - Logs differences and attempts repair when configured.

11. MIGRATION AGENT ARCHITECTURE

Agent Installation:

- User runs a script on a VPS:
 - Example:

```
curl -s https://panel.example.com/install-agent.sh | bash -s -- --server https://panel.example.com --token AGENT_TOKEN
```
- Script:
 - Downloads agent binary or Docker image.
 - Writes config with server URL and agent token.
 - Installs as a systemd service.

Agent Behavior:

- On startup:
 - Registers with backend using the token.

- Receives a unique agent id.
- Main loop:
 - Sends heartbeat regularly.
 - Polls /next-job endpoint for assigned work.
 - Executes migration jobs with multi-threaded R2 copy logic.
 - Reports progress and events.
 - On completion:
 - Reports success or failure.

Multi-Agent Support:

- Multiple agents can run in parallel across different VPS servers.
- Large migrations are split into sub-jobs (e.g., by key prefix).
- Backend assigns sub-jobs to available agents, balancing load.

12. RECURRING 20-DAY AUTO-MIGRATION

Auto-Migration Logic:

- A scheduled process runs periodically (e.g., every hour).
- For each account in cf_accounts:
 - If auto_migrate_enabled = true AND status = "active"
 - AND (last_migrated_at is NULL OR now - last_migrated_at >= migrate_interval_days)
 - THEN:
 - Create a migration_job from this account to migrate_to_account_id.
 - Set last_migrated_at to the job start time or completion time based on design.

Post-Migration Actions:

- When verification passes:
 - Set source account status = "disabled"
 - Set destination account status = "active"
 - For all buckets and spaces referencing the old account:
 - Update them to point to the new account or equivalent destination buckets.
 - Log an audit entry recording the account switch.

Manual Controls:

- Admin can:
 - Disable or enable auto-migration per account.
 - Change migration interval.
 - Change migration target account.
 - Manually trigger migration outside of schedule.

13. DATA INTEGRITY & ACCURACY STRATEGY

Objective:

Achieve practical 100% accuracy for migrations and operations, with transparency of any deviations.

Key Techniques:

1) Double-Phase Migration (Copy + Verify)

- Phase 1: Copy all objects from source to destination.
- Phase 2: Verify:
 - Object count equality.
 - Total bytes equality.
 - Optional per-object match (size and ETag) for all or sampled objects.

2) Repair Sub-Jobs

- If mismatches detected:
 - Create repair jobs to recopy missing or mismatched objects.
 - Re-run verification on affected subset.

3) Idempotent Design

- Migration jobs and sub-jobs can be safely re-run.
- Copy operation only overwrites when objects differ or when forced.

4) Checkpoints

- For large sets, store last processed key/prefix to resume on crash or restart.

5) Reporting

- Migration job exposes detailed results in the UI.
- Admins can view mismatch lists and repair logs.

The system does not mark the migration as "final" until verification passes or the admin explicitly accepts known, documented discrepancies.

14. ADVANCED SEARCH & FILTERS

Search Capabilities:

- Search by file name (full or partial).
- Search by path/prefix.
- Filter by:
 - account
 - bucket
 - size range
 - content type (image, video, audio, document, archive)
 - created/modified date range
 - owner user
 - public/private status

Example Search API:

GET /api/search?query=report&bucket=logs&minSize=1048576&maxSize=1073741824&type=pdf&ov

Search Behavior:

- Uses an indexed objects table for performance.
- Results show:
 - File name, path, bucket, account, size, date, owner, quick actions (open, copy link).

15. AUDIT LOGGING & ACTIVITY FEED

Audit Logging:

- Every important action writes an entry into audit_logs:
 - User login/logout
 - File upload/download/delete
 - Sharing/unsharing links
 - Migration creation/completion/failure
 - Account status changes
 - Permission changes
- Each log has:
 - user_id
 - action string
 - target_type (e.g. "object", "bucket", "account", "user", "migration_job")
 - target_id
 - timestamp
 - details_json

Activity Feed:

- Per-user feed showing their recent actions.
- Global admin feed showing overall activity across the system.

16. SECURITY MODEL

Authentication:

- Email/password (hashed).
- Optional MFA for admin users.
- Session-based or JWT-based tokens.

Authorization:

- Role-based access control.
- Bucket-level permissions.
- Object-level access through space/bucket relationships.

Secrets Management:

- Cloudflare credentials stored encrypted in DB.

- Agent tokens stored encrypted.
- Decryption keys in environment variables or external KMS.

API Security:

- Rate-limiting on login and public endpoints.
- CSRF protection on web UI.
- HTTPS enforced end-to-end.

Link Security:

- Public links include long, random tokens.
- Optional password and expiry for shared links.
- Signed links use backend-generated HMAC or R2 pre-signed URLs.

17. FAILURE SCENARIOS & RECOVERY

Potential Failures:

1) Network interruptions during migration

- Recovery: resume from last checkpoint and retry chunks/objects.

2) Agent crash or VPS outage

- Recovery: job remains in "running" but with heartbeat timeout; backend reassigns remaining work to another agent.

3) Cloudflare API errors or throttling

- Recovery: exponential backoff and retry logic; log warnings.

4) Data mismatches on verification

- Recovery: repair sub-jobs recopy problematic objects; admin view shows details.

5) Database outage

- Recovery: run migrations with transaction safety; restart worker when DB is back; do not lose state of jobs already persisted.

18. MONITORING & METRICS

Metrics to Track:

- Number of active agents and their load.
- Number of migration jobs (pending, running, verifying, completed, failed).
- Transfer throughput (MB/s) per job and per agent.
- Error rates on API calls (Cloudflare, DB, etc.).
- Storage usage per user, per space, per bucket, per account.

Dashboards:

- Admin dashboard showing high-level stats and alerts.
- Migration dashboard showing active and historical jobs.
- Agent dashboard showing agent health, last seen timestamps, and assigned jobs.

19. DOCUMENTATION PAGE (WEBSITE)

The panel will include a Documentation section accessible via a “Help” or “Docs” link in the UI.

Sections:

1) Getting Started

- Creating your admin account.
- Connecting your first Cloudflare account.
- Creating buckets and spaces.

2) Managing Files

- Uploading files.
- Creating folders.
- Moving and renaming files.
- Deleting and restoring from trash.

3) Sharing Files

- Creating public share links.
- Setting expiry and passwords.
- Revoking shared links.

4) Managing Accounts & Buckets

- Adding Cloudflare accounts.
- Understanding active vs disabled status.
- Setting up automatic migrations.
- Assigning buckets to users.

5) Migrations

- Direct vs Cloudflare migration modes.
- Installing and registering agents on VPS.
- Monitoring progress, verifying data.
- Auto-migrations every 20 days.

6) URL Endpoints

- /f/:account/:bucket/:path – file viewer.
- /raw/:account/:bucket/:path – raw access.
- /p/:token – public share.

7) API Usage

- Authentication.
- Examples for listing files, uploading, searching, and producing share links.

8) Troubleshooting

- Common migration errors.

- Agent connectivity issues.
- Quota and permission errors.

9) FAQ

- How secure is my data?
- What happens during auto-migration?
- How do I scale migrations for 20+ TB?

20. API REFERENCE (SUMMARY)

Examples (not exhaustive):

Accounts:

- GET /api/accounts
- POST /api/accounts
- PATCH /api/accounts/:id
- POST /api/accounts/:id/migrate

Buckets:

- GET /api/accounts/:id/buckets
- POST /api/accounts/:id/buckets
- DELETE /api/buckets/:id
- PATCH /api/buckets/:id/public

Objects:

- GET /api/buckets/:id/objects
- POST /api/objects/upload
- DELETE /api/objects/:id
- POST /api/objects/:id/share

Users:

- GET /api/users
- POST /api/users
- PATCH /api/users/:id

Search:

- GET /api/search

Migrations:

- GET /api/migrations
- POST /api/migrations
- GET /api/migrations/:id

Agents:

- POST /api/agents/register
- POST /api/agents/heartbeat
- POST /api/agents/next-job
- POST /api/agents/progress
- POST /api/agents/job-complete

21. FUTURE ENHANCEMENTS

- WebDAV or S3-compatible front-end API for third-party apps.
- Built-in media transcoding and image thumbnailing.
- Virus scanning integration.
- Lifecycle rules and cold storage tiers.
- Multi-cloud provider support beyond R2 (AWS, GCP, Backblaze).
- Billing and subscription system for multi-tenant SaaS usage.

End of Master Document.